Individual Submission: ML Workflow with DVC

Author: Ali Cihan Ozdemir (ID: 9091405)

Hardware Note

Training in this project is optimized for Apple Silicon hardware by using PyTorch's Metal Performance Shaders (MPS) backend when it is available. This design reduces training latency on modern Mac systems while preserving a CPU fallback for reproducibility on machines that do not expose an MPS device.

ML Workflow with DVC

This notebook is a guided workshop for building a minimal, reproducible machine learning workflow with **Data Version Control (DVC)** using the project structure below.

```
project/
├── data/
├── src/
│   ├── prepare.py
│   ├── train.py
├── params.yaml
├── dvc.yaml
```

The workflow assumes you are working in Visual Studio Code and running commands from the project root folder.

1. Introduction to DVC

DVC stands for Data Version Control.
It is an open-source tool designed for managing machine learning and data science projects, especially when you work with large datasets, models, and experiments that don't fit well into Git alone.

In the workplace, DVC is useful when teams need to coordinate code, data, models, and results without placing large binary assets directly inside a Git repository. In many organizations, software engineers, data scientists, and researchers work together across branches, machines, and cloud environments. DVC helps these teams keep a lightweight Git history while storing the actual datasets and trained models in external storage. This makes it easier to reproduce results, compare experiments, onboard new team

members, and reduce confusion about which dataset or model version produced a particular result. DVC is especially valuable in research labs, applied ML teams, and MLOps pipelines where traceability and repeatability matter.

Official DVC portal: https://dvc.org/
Official documentation: https://doc.dvc.org/

2. What is the role of DVC in the MLOps workflow?

DVC sits at the intersection of **code versioning, data management, experiment tracking, and reproducibility**. In a modern MLOps workflow, it helps connect what was run, on which data, with which parameters, and what outputs were produced.

What problem does DVC solve?

Git works very well for **code**, but machine learning projects often include assets that Git alone does not handle elegantly:

- Large datasets
- Model checkpoints and trained models
- Intermediate artifacts
- Repeatable experiment pipelines
- Clear traceability between code, data, and results

DVC complements Git by versioning the **metadata** about large files and pipeline outputs while storing the heavy artifacts elsewhere.

What DVC does

DVC helps you:

✅ Version datasets and models

- Keeps lightweight metadata (`.dvc` files) in Git
- Stores actual data in external storage (local disk, S3, Azure Blob, GCP, SSH, etc.)

✅ Track experiments

- Records parameters, metrics, and outputs
- Allows you to compare experiments (like a Git diff for ML)

✅ Ensure reproducibility

- Defines pipelines so models can be rebuilt from raw data
- Tracks dependencies between data, code, and models

✅ Collaborate effectively

- Any evaluator can reproduce results exactly
- Everyone gets the same data/model versions tied to a Git commit

3. Build a Minimal DVC Project

In this section, students build and run a small DVC project for a CNN trained on MNIST.

Learning goals

By the end of this section, students should be able to:

- initialize a Git + DVC project
- understand the role of each project file
- run a two-stage DVC pipeline
- inspect metrics
- make a controlled experiment change
- explain why DVC reruns some stages and skips others

3.1 Setup commands (Bash)

In [15]:
```
# Run these in a Bash terminal from the project root

# pip install dvc torch torchvision scikit-learn pandas pyyaml
# git init
# dvc init
```

Explanation

- `pip install ...` installs the Python packages used in this workshop.
- `git init` creates a Git repository so code and DVC metadata can be versioned.
- `dvc init` creates the DVC configuration inside the repository.

3.2 Setup commands (PowerShell)

In [16]:
```
# Run these in PowerShell from the project root

# pip install dvc torch torchvision scikit-learn pandas pyyaml
# git init
# dvc init
```

The commands are the same in PowerShell for this basic setup. Later, file paths may look slightly different on Windows.

3.3 Project structure

```
project/
├── data/
├── src/
│   ├── prepare.py
│   ├── train.py
├── params.yaml
├── dvc.yaml
```

What each part does

- `data/` stores raw and processed data artifacts.
- `src/prepare.py` downloads and prepares the dataset.
- `src/train.py` trains the CNN and writes outputs for the pipeline.
- `params.yaml` stores experiment settings such as epochs and learning rate.
- `dvc.yaml` defines the DVC pipeline stages, their dependencies, and their outputs.

3.4 Create `params.yaml`

```yaml
epochs: 2
lr: 0.001
batch_size: 64
```

This file stores experiment parameters outside the training code. That makes it easier to rerun the pipeline after changing one setting and observe the result.

3.5 `src/prepare.py`

In [17]:
```python
import os
import torch
from torchvision import datasets, transforms

OUTPUT_DIR = "data/processed"
os.makedirs(OUTPUT_DIR, exist_ok=True)

transform = transforms.Compose([
    transforms.ToTensor()
])

train_dataset = datasets.MNIST(
    root="data/raw",
    train=True,
    download=True,
    transform=transform
)

test_dataset = datasets.MNIST(
    root="data/raw",
    train=False,
    download=True,
```

```
    transform=transform
)

def dataset_to_tensors(dataset):
    images = []
    labels = []

    for img, label in dataset:
        images.append(img)
        labels.append(label)

    images = torch.stack(images)
    labels = torch.tensor(labels)

    return images, labels

train_images, train_labels = dataset_to_tensors(train_dataset)
test_images, test_labels = dataset_to_tensors(test_dataset)

torch.save((train_images, train_labels), os.path.join(OUTPUT_DIR, "train.pt"
torch.save((test_images, test_labels), os.path.join(OUTPUT_DIR, "test.pt"))

print("Data preparation complete.")
print(f"Saved to: {OUTPUT_DIR}")
```

```
Data preparation complete.
Saved to: data/processed
```

Role of `prepare.py` in the pipeline

This script is the **data preparation stage**. It:

- downloads MNIST
- converts the dataset into tensors
- saves the processed training and test data to disk

For teaching purposes, this script is intentionally simple. It avoids complex preprocessing so students can focus on the workflow and the role of DVC.

3.6 `src/train.py`

In [18]:
```
import os
import json
import yaml
import torch
import torch.nn as nn
import torch.optim as optim

DATA_DIR = "data/processed"
MODEL_PATH = "model.pt"
METRICS_PATH = "metrics.json"
```

```python
with open("params.yaml", "r", encoding="utf-8") as f:
    params = yaml.safe_load(f)

EPOCHS = params["epochs"]
LR = params["lr"]
BATCH_SIZE = params["batch_size"]

train_images, train_labels = torch.load(os.path.join(DATA_DIR, "train.pt"))
test_images, test_labels = torch.load(os.path.join(DATA_DIR, "test.pt"))

class SimpleCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv = nn.Conv2d(1, 8, kernel_size=3)
        self.pool = nn.MaxPool2d(2)
        self.fc = nn.Linear(8 * 13 * 13, 10)

    def forward(self, x):
        x = self.pool(torch.relu(self.conv(x)))
        x = x.view(x.size(0), -1)
        x = self.fc(x)
        return x

model = SimpleCNN()
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=LR)

model.train()
for epoch in range(EPOCHS):
    for i in range(0, len(train_images), BATCH_SIZE):
        x_batch = train_images[i:i + BATCH_SIZE]
        y_batch = train_labels[i:i + BATCH_SIZE]

        optimizer.zero_grad()
        outputs = model(x_batch)
        loss = criterion(outputs, y_batch)
        loss.backward()
        optimizer.step()

    print(f"Epoch {epoch + 1}/{EPOCHS}, Loss: {loss.item():.4f}")

model.eval()
correct = 0
total = 0

with torch.no_grad():
    outputs = model(test_images)
    _, predicted = torch.max(outputs, 1)
    total += test_labels.size(0)
    correct += (predicted == test_labels).sum().item()

accuracy = correct / total
print(f"Test Accuracy: {accuracy:.4f}")

torch.save(model.state_dict(), MODEL_PATH)
```

```python
metrics = {"accuracy": round(accuracy, 4)}
with open(METRICS_PATH, "w", encoding="utf-8") as f:
    json.dump(metrics, f, indent=4)

print(f"Model saved to {MODEL_PATH}")
print(f"Metrics saved to {METRICS_PATH}")
```

```
Epoch 1/5, Loss: 0.1247
Epoch 2/5, Loss: 0.0672
Epoch 3/5, Loss: 0.0370
Epoch 4/5, Loss: 0.0304
Epoch 5/5, Loss: 0.0318
Test Accuracy: 0.9657
Model saved to model.pt
Metrics saved to metrics.json
```

Role of `train.py` in the pipeline

This script is the **training stage**. It:

- loads the processed tensors
- reads hyperparameters from `params.yaml`
- trains a tiny CNN
- saves a trained model to `model.pt`
- writes evaluation results to `metrics.json`

In this workshop, the model is intentionally small so that students can run it quickly on a CPU and focus on reproducibility rather than model complexity.

3.7 `dvc.yaml`

```yaml
stages:
  prepare:
    cmd: python src/prepare.py
    deps:
      - src/prepare.py
    outs:
      - data/processed

  train:
    cmd: python src/train.py
    deps:
      - src/train.py
      - data/processed
    params:
      - epochs
      - lr
      - batch_size
    outs:
      - model.pt
```

```yaml
    metrics:
      - metrics.json
```

How to read `dvc.yaml`

- `stages` defines the pipeline.
- `prepare` is the first stage and runs `python src/prepare.py`.
- `deps` lists dependencies. If one changes, DVC knows the stage may need to rerun.
- `outs` lists outputs produced by a stage.
- `train` depends on both `src/train.py` and the prepared data directory.
- `params` tells DVC which experiment settings should be tracked from `params.yaml`.
- `metrics` tells DVC which result files should be displayed and compared.

This file is the heart of the pipeline: it defines **what runs, in what order, based on which inputs**.

3.8 Run the pipeline

In [19]:
```bash
# Bash

!/opt/homebrew/Caskroom/miniconda/base/bin/python src/prepare.py
!/opt/homebrew/Caskroom/miniconda/base/bin/python src/train.py
```

```
Data preparation complete.
Saved to: data/processed
First batch outputs sample: [[-0.3790629208087921, 0.11074042320251465, 0.18
116934597492218, 0.1128324493765831, 0.3947363495826721, -0.1280335336923599
2, 0.23604901134967804, -0.006865097209811211, 0.04563520848751068, -0.11603
12220454216], [-0.28470394015312195, 0.0711887925863266, 0.2656288146972656,
0.08507103472948074, 0.3353150188922882, -0.13868893682956696, 0.23455440998
077393, -0.05465111881494522, -0.07219763100147247, -0.08397141098976135],
[-0.22341188788414001, 0.15979605913162231, 0.12747663259506226, 0.070943772
79281616, 0.3400968909263611, -0.04900269955396652, 0.2554638981819153, -0.1
5262679755687714, -0.012499464675784111, -0.05429980903863907], [-0.16761392
35496521, 0.10432282090187073, 0.02235373854637146, 0.06264270097017288, 0.3
621649742126465, -0.019522806629538536, 0.19813236594200134, -0.021983476355
671883, 0.08453020453453064, -0.07456552237272263], [-0.28012973070144653,
0.052495431154966354, 0.1309613734483719, 0.06667269021272659, 0.42943972349
16687, -0.22604019939899445, 0.22195646166801453, -0.17242993414402008, 0.10
111536830663681, -0.1641855537891388]]
First batch loss: 2.328554391860962
First batch conv gradient norm: 0.11832902580499649
Epoch 1/5, Loss: 0.1196
Epoch 2/5, Loss: 0.0698
Epoch 3/5, Loss: 0.0493
Epoch 4/5, Loss: 0.0373
Epoch 5/5, Loss: 0.0262
Device: mps
Test Accuracy: 0.9679
Model saved to model.pt
Metrics saved to metrics.json
```

Optional warm-up: run the scripts directly once so students can see what each stage does individually.

In [20]:

```
# Bash

!/opt/homebrew/Caskroom/miniconda/base/bin/python -m dvc repro
!/opt/homebrew/Caskroom/miniconda/base/bin/python -m dvc metrics show
```

```
Stage 'prepare' didn't change, skipping
Stage 'train' didn't change, skipping
Stage 'predict' didn't change, skipping
Data and pipelines are up to date.
Path          accuracy    loss
metrics.json  0.9679      0.02623
```

```
# PowerShell

dvc repro
dvc metrics show
```

What students should notice

- DVC runs the stages in dependency order.
- The pipeline creates `data/processed`, `model.pt`, and `metrics.json`.
- `dvc metrics show` reads the accuracy directly from the metrics file.

3.9 Example metrics file

```
{
  "accuracy": 0.6967
}
```

3.10 Make controlled changes to the experiment

The idea is to change **one thing at a time** and then rerun the pipeline.

Example A: Increase the number of epochs

```
epochs: 5
lr: 0.001
batch_size: 64
```

In [21]:
```
# Bash

!/opt/homebrew/Caskroom/miniconda/base/bin/python -m dvc repro
!/opt/homebrew/Caskroom/miniconda/base/bin/python -m dvc metrics show
!/opt/homebrew/Caskroom/miniconda/base/bin/python -m dvc metrics diff
```

```
Stage 'prepare' didn't change, skipping
Stage 'train' didn't change, skipping
Stage 'predict' didn't change, skipping
Data and pipelines are up to date.
Path          accuracy    loss
metrics.json  0.9679      0.02623
```

Example B: Change the learning rate

```
epochs: 2
lr: 0.0005
batch_size: 64
```

In [22]:
```
# Bash

!/opt/homebrew/Caskroom/miniconda/base/bin/python -m dvc repro
!/opt/homebrew/Caskroom/miniconda/base/bin/python -m dvc metrics show
!/opt/homebrew/Caskroom/miniconda/base/bin/python -m dvc metrics diff
```

```
Stage 'prepare' didn't change, skipping
Stage 'train' didn't change, skipping
Stage 'predict' didn't change, skipping
Data and pipelines are up to date.
Path          accuracy    loss
metrics.json  0.9679      0.02623
```

Example C: Change the batch size

```
epochs: 2
lr: 0.001
batch_size: 128
```

In [23]:
```
# Bash

!/opt/homebrew/Caskroom/miniconda/base/bin/python -m dvc repro
!/opt/homebrew/Caskroom/miniconda/base/bin/python -m dvc metrics show
!/opt/homebrew/Caskroom/miniconda/base/bin/python -m dvc metrics diff
```

```
Stage 'prepare' didn't change, skipping
Stage 'train' didn't change, skipping
Stage 'predict' didn't change, skipping
Data and pipelines are up to date.
Path          accuracy    loss
metrics.json  0.9679      0.02623
```

Example D: Re-run with no changes

`dvc repro`

Students should observe that DVC skips unchanged stages. This is a strong demonstration of dependency-aware workflow execution.

3.11 Save the project state in Git

In [24]:
```
# Bash

!git add data/.gitignore .gitignore dvc.yaml dvc.lock params.yaml src/prepar
!git commit -m "Add minimal DVC MNIST pipeline"
```

```
The following paths are ignored by one of your .gitignore files:
metrics.json
hint: Use -f if you really want to add them.
hint: Disable this message with "git config set advice.addIgnoredFile false"
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   MK_Workflow_DVC.ipynb

no changes added to commit (use "git add" and/or "git commit -a")
```

```
# PowerShell

!git add data\.gitignore .gitignore dvc.yaml dvc.lock params.yaml
src/prepare.py src/train.py metrics.json
!git commit -m "Add minimal DVC MNIST pipeline"
```

4. Talking Points

Use these during the workshop discussion.

💡 Talking Point 1

This pipeline is intentionally minimal. The goal is to teach workflow concepts, not advanced model design.

💡 Talking Point 2

DVC works best when code, parameters, data, and outputs are treated as connected parts of one reproducible system.

💡 Talking Point 3

Changing a parameter is not just “running training again.” It becomes a tracked, comparable experiment.

💡 Talking Point 4

When DVC skips a stage, it is showing students that reproducibility is also about **efficiency**.

💡 Talking Point 5

A weak result in a simple model is still educationally valuable if the workflow is transparent and reproducible.

💡 Talking Point 6

This small project mirrors a larger professional pattern: data preparation, model training, metrics, and versioned pipeline logic.

💡 Talking Point 7

Students often think reproducibility starts after model selection. In practice, it starts the moment raw data enters the workflow.

💡 Talking Point 8

DVC does not replace Git. It extends Git-based collaboration into the data and model parts of machine learning work.

🗣️ Additional Talking Points for Section 5

0. Important subtle issue (VERY useful for teaching)

Importing a class from another Python file only works cleanly when that file separates **definitions** from **execution**. If `train.py` runs training code as soon as it is imported, then importing `SimpleCNN` inside prediction code may accidentally trigger training again.

1. Best practice (recommended fix)

A good pattern is to place the training workflow inside a `main()` function and then run it only with:

```
if __name__ == "__main__":
    main()
```

This allows other files to import `SimpleCNN` safely without causing the training script to execute.

2. Teaching insight (very valuable moment)

This is a useful example of a broader software engineering principle in machine learning: separate **model definitions** from **script execution**. Students often write everything in one file at first, but reusable ML workflows benefit from modular code.

3. Important note about imports in the notebook vs the pipeline

The prediction example in **Section 5.1** can run in the notebook with:

```
from src.train import SimpleCNN
```

because the notebook is usually executed from the project root, where `src` is visible as a package-like folder.

However, this will not necessarily run as-is when the same code is saved to `src/predict.py` and executed from the pipeline with:

```
python src/predict.py
```

For that script-based example, the import may need to be refactored to:

```
from train import SimpleCNN  # Assuming train.py is in the same directory for this example
```

This distinction is worth highlighting explicitly: **execution context affects how imports work**.

5. Expand the Pipeline: Add a Classification Stage

So far, the pipeline prepares data and trains a CNN. A natural next step is to **use the trained model to run classifications** and save prediction outputs.

In this extension, students add a new script called `src/predict.py`. This script loads:

- the trained model from `model.pt`
- the processed test data from `data/processed/test.pt`

It then runs inference and saves a small results file. This demonstrates an important MLOps idea: **training is not the end of the workflow**. In practice, models are trained so they can be used for prediction, evaluation, and downstream decision-making.

Updated pipeline idea

```
prepare  →  train  →  predict
```

The new `predict` stage depends on both the trained model and the prediction script. DVC will only rerun this stage when one of its dependencies changes.

This is a good example of how pipelines grow over time: first data preparation, then training, then inference.

5.1 Create `src/predict.py`

```python
import json
import os
import torch

# Import the model class from train.py
# IMPORTANT: Ensure that the SimpleCNN class is defined in
src/train.py and is accessible here
# and also when the code is run from a Python script, the current
working directory should be the project root
from src.train import SimpleCNN

DATA_PATH = os.path.join("data", "processed", "test.pt")
MODEL_PATH = "model.pt"
OUTPUT_PATH = "predictions.json"

# Load data
test_images, test_labels = torch.load(DATA_PATH)

# Instantiate model
model = SimpleCNN()

# Load trained weights
```

```
model.load_state_dict(torch.load(MODEL_PATH))
model.eval()
```

5.2 Update `dvc.yaml`

Add a third stage to the pipeline:

```yaml
predict:
  cmd: python src/predict.py
  deps:
    - src/predict.py
    - model.pt
    - data/processed/test.pt
  outs:
    - predictions.json
```

With this addition, DVC understands that prediction depends on the trained model and the processed test data. If the model changes because training is rerun, the `predict` stage will also rerun.

5.3 Update `src/predict.py`

Add instruction to run inferences

```python
import json
import os
import torch

# Import the model definition from train.py
# IMPORTANT: Ensure that the SimpleCNN class is defined in
src/train.py and is accessible here
# and also when the code is run from a Python script, the current
working directory should be the project root
from src.train import SimpleCNN

DATA_PATH = os.path.join("data", "processed", "test.pt")
MODEL_PATH = "model.pt"
OUTPUT_PATH = "predictions.json"

# Load data
test_images, test_labels = torch.load(DATA_PATH)

# Initialize model and load weights
model = SimpleCNN()
model.load_state_dict(torch.load(MODEL_PATH))
model.eval()

# TODO: Add code to run inference and save predictions
with torch.no_grad():
    outputs = model(test_images)
```

```
    predicted = torch.argmax(outputs, dim=1)

# Save a small sample of predictions
sample_results = []
for i in range(10):
    sample_results.append({
        "index": i,
        "true_label": int(test_labels[i].item()),
        "predicted_label": int(predicted[i].item())
    })

with open(OUTPUT_PATH, "w", encoding="utf-8") as f:
    json.dump(sample_results, f, indent=4)

print(f"Saved predictions to {OUTPUT_PATH}")
```

5.3 Run the expanded pipeline

Once `src/predict.py` and the updated `dvc.yaml` are in place, students can rerun the pipeline from the project root.

They should observe that DVC now executes three stages:

- `prepare`
- `train`
- `predict`

and produces a new output file:

- `predictions.json`

That file contains a small sample of predicted labels from the trained model.

In [25]:
```
# Bash

!/opt/homebrew/Caskroom/miniconda/base/bin/python -m dvc repro
!cat predictions.json
```

```
Stage 'prepare' didn't change, skipping
Stage 'train' didn't change, skipping
Stage 'predict' didn't change, skipping
Data and pipelines are up to date.
[
    {
        "index": 0,
        "true_label": 7,
        "predicted_label": 7
    },
    {
        "index": 1,
        "true_label": 2,
        "predicted_label": 2
    },
    {
        "index": 2,
        "true_label": 1,
        "predicted_label": 1
    },
    {
        "index": 3,
        "true_label": 0,
        "predicted_label": 0
    },
    {
        "index": 4,
        "true_label": 4,
        "predicted_label": 4
    },
    {
        "index": 5,
        "true_label": 1,
        "predicted_label": 1
    },
    {
        "index": 6,
        "true_label": 4,
        "predicted_label": 4
    },
    {
        "index": 7,
        "true_label": 9,
        "predicted_label": 9
    },
    {
        "index": 8,
        "true_label": 5,
        "predicted_label": 6
    },
    {
        "index": 9,
        "true_label": 9,
        "predicted_label": 9
    }
]
```

```
# PowerShell

!dvc repro
!Get-Content predictions.json
```

5.4 Try a follow-up experiment

Change a value in `params.yaml`, rerun the pipeline, and inspect whether the predictions change.

For example:

- increase `epochs`
- change `lr`
- change `batch_size`

Then run:

```
dvc repro
dvc metrics show
```

Students should notice that once training changes, the prediction stage also reruns because it depends on `model.pt`.

This reinforces a key workflow lesson: **pipeline stages are connected by dependencies, not by manual memory**.

6. ⚔️ Challenges

These challenges extend the workshop by connecting the DVC-based CNN pipeline to core neural network concepts covered elsewhere in the course. Each challenge includes a coding task and a reflection task.

For each challenge:

1. complete the coding task
2. run the relevant part of the workflow
3. write **at least three talking points** that explain what you observed and why it matters

Your talking points should focus on reasoning, not just reporting a number.

6.1 ⛰️ Challenge: Hyperparameters, Activation Functions, and Initialization

This challenge connects the CNN workflow to activation-function choice, vanishing gradients, and initialization strategy. The slide deck emphasizes comparing gradients for

sigmoid and ReLU-family functions, and notes that ReLU is often a strong default for simple tasks while GELU can be stronger on more complex tasks. It also highlights the role of fan-in/fan-out and Glorot/Xavier initialization in stable training.

Coding task

Modify `src/train.py` so that the model can be trained with **different activation functions**. For example:

- `ReLU`
- `LeakyReLU`
- `GELU`

Then add an activation setting to `params.yaml`, such as:

```
activation: relu
```

Update the model code so the activation is chosen from the parameter file. After that, run at least **three experiments**, one per activation function.

Suggested extension

Add a second parameter for initialization strategy and test one of these:

- default PyTorch initialization
- Xavier/Glorot initialization
- He/Kaiming initialization

Commands to run

```
dvc repro
dvc metrics show
dvc metrics diff
```

Deliverable

Submit:

- the updated code
- the parameter settings you tried
- the resulting metrics

Talking points

Write **at least three talking points** that address questions such as:

- Why might sigmoid-like behavior contribute to vanishing gradients in deeper networks?
- Why is ReLU often a practical default?
- In what situations might GELU or LeakyReLU be worth the extra complexity?

- How can initialization affect optimization stability and training speed?

6.2 ⛰️ Challenge: Optimizers

This challenge focuses on how optimization changes learning dynamics. The slide deck covers gradients, momentum, Nesterov acceleration, AdaGrad, RMSProp, and Adam, and emphasizes that Adam combines momentum-like behavior with adaptive scaling.

Coding task

Modify `src/train.py` so that the optimizer is selected from `params.yaml`. Start by supporting at least:

- `SGD`
- `SGD with momentum`
- `Adam`

Example parameter additions:

```
optimizer: adam
momentum: 0.9
```

Then run at least **three experiments** with the same model and dataset, changing only the optimizer settings. Keep the other hyperparameters fixed so your comparison is meaningful.

Suggested extension

If time permits, compare two learning rates for one optimizer and observe whether the optimizer is robust or sensitive to that choice.

Commands to run

```
dvc repro
dvc metrics show
dvc metrics diff
```

Deliverable

Submit:

- the updated optimizer-selection code
- the parameter settings you used
- a short comparison of performance and training behavior

Talking points

Write **at least three talking points** that address questions such as:

- How does momentum change the behavior of plain gradient descent?

- Why might Adam converge faster or more smoothly in this project?
- Why is it important to compare optimizers while holding the rest of the setup constant?
- When would a simpler optimizer still be preferable?

6.3 ⛰️ Challenge: Forward and Backward Passes

This challenge connects your CNN workflow to the mechanics of neural network learning. The slide deck walks through a small network, its forward pass, the resulting loss, and how changes in weights affect the loss during backpropagation.

Coding task

Add instrumentation to `src/train.py` so that, for one mini-batch, you inspect both the **forward pass** and the **backward pass**.

Your code should do all of the following:

1. print or log the model outputs for one batch before the optimizer step
2. compute the loss for that batch
3. call `loss.backward()`
4. inspect at least one gradient tensor, such as:
   - convolution weights
   - fully connected weights
5. perform the optimizer step
6. optionally print the norm of one parameter before and after the update

A simple example of the kind of inspection you might add is:

```
print(outputs[:5])
print(loss.item())
print(model.conv.weight.grad.norm().item())
```

Suggested extension

Save selected gradient statistics to a JSON file so that they become part of the reproducible workflow.

Commands to run

```
dvc repro
dvc metrics show
```

Deliverable

Submit:

- the instrumented training code

- a short explanation of what happened during one forward and backward pass
- any gradient values, norms, or observations you recorded

Talking points

Write **at least three talking points** that address questions such as:

- What information is produced in the forward pass?
- What role does the loss play between the forward and backward passes?
- What does a gradient tell you about a parameter update?
- How do forward and backward passes connect the mathematics of learning to the code you executed?

6.4 Optional submission format

For each challenge, you may organize your response as:

- **Code changes**
- **Results**
- **At least three talking points**

This format keeps the work aligned with the goals of the workshop: build, run, observe, and explain.

6.1 Individual Talking Points

- In my implementation, ReLU is a strong baseline because it preserves large positive signals without saturating, which helps gradients remain usable during backpropagation. This matters mathematically because saturated activations shrink local derivatives and reduce the magnitude of updates in earlier layers.
- LeakyReLU is valuable when I want to reduce the risk of permanently inactive neurons. By assigning a small slope to negative inputs, it keeps a non-zero derivative on that side of the function and therefore maintains a gradient path even when activations fall below zero.
- GELU is academically interesting because it applies a smooth gating effect instead of a hard threshold. That smoother nonlinearity can produce more nuanced feature selection, although it may introduce slightly higher computational cost than ReLU in a simple CNN like this one.

6.2 Individual Talking Points

- Plain SGD updates parameters directly along the negative gradient, so it is easy to interpret but can converge slowly when the loss surface contains narrow valleys or

oscillatory directions. Its simplicity is useful when I want a transparent baseline for comparison.

- Adding momentum changes the update rule by incorporating a moving average of past gradients. In practice, this helps accelerate movement in consistent directions and dampens zig-zag behavior, which is why momentum-based SGD often reaches a stable region faster than vanilla SGD.
- Adam usually converges faster at the beginning of training because it combines momentum-like behavior with per-parameter adaptive learning rates. The reason this matters is that different weights can experience different gradient scales, and Adam rescales updates automatically instead of forcing one fixed step size on every parameter.

6.3 Individual Talking Points

- The forward pass converts an input image into logits by applying learned linear filters and nonlinear transformations. From a mathematical perspective, it is the evaluation of a composed function whose output is then compared with the true label through the loss function.
- The loss is the bridge between prediction and learning because it converts model error into a scalar objective that can be differentiated. Once `loss.backward()` is called, PyTorch uses the chain rule to compute how much each parameter contributed to the final error.
- The gradient norm of `model.conv.weight` summarizes the magnitude of the update signal reaching the first convolutional layer. A larger norm means the loss is currently more sensitive to those weights, while a very small norm can indicate weak learning signal, saturation, or a vanishing-gradient tendency.

7. Individual Submission Notes

This notebook documents the individual CSCN8010 submission completed by Ali Cihan Ozdemir. The final deliverable aligns the DVC pipeline, PyTorch implementation, challenge reflections, and repository documentation with an individual MLOps workflow rather than a shared workshop exercise.